

Solutionnaire du TP2

À votre tour

Charger le fichier

```
objet1 <- read.csv('/home/public/R/TP2.csv')
```

Le résultat doit être **assigné** à une variable, sinon il est seulement affiché dans la console. Vérifiez le chargement avec `head(objet1)`, `dim(objet1)` ou le panneau *Environment*.

0. Extraire un *rectangle* de données

On sélectionne des rangées **et** des colonnes consécutives avec deux vecteurs d'indices `[rangées, colonnes]`. Par exemple, les rangées 2 et 3 sur les colonnes 1 à 3 :

```
objet1[2:3, 1:3]
#>   CHROM   POS      ID
#> 2     1 10611 rs189107123
#> 3     1 13302 rs180734498
```

1. Les noms de colonnes

```
noms_colonnes <- colnames(objet1)
noms_colonnes[1:8]
#> [1] "CHROM" "POS" "ID" "REF" "ALT" "QUAL" "FILTER" "FORMAT"
```

2. La colonne **ID**

Plusieurs écritures sont possibles. `$` et `[, "ID"]` renvoient un **vecteur**, alors que `["ID"]` renvoie un **data.frame** d'une seule colonne :

```
ids <- objet1$ID           # vecteur
ids <- objet1[, "ID"]      # vecteur (équivalent)
class(objet1["ID"])        # data.frame d'une colonne
#> [1] "data.frame"
```

3. Le score qualité du SNP **rs189107123**

On compare chaque identifiant au SNP voulu avec `==`, ce qui donne un vecteur logique (un seul **TRUE**), puis on l'utilise pour sélectionner la rangée :

```
rangee <- ids == "rs189107123"
objet1[rangee, "QUAL"]
#> [1] 100
```

4. La position du SNP **rs4030331**

Même principe que la question 3, sur la colonne **POS** :

```
objet1[ids == "rs4030331", "POS"]
#> [1] 82734
```

5. Le score qualité moyen

```
mean(objet1$QUAL)
#> [1] 110.16
```

6. Les SNPs de qualité supérieure à 99

La condition `objet1$QUAL > 99` sélectionne les rangées ; le nom **"ID"** sélectionne la colonne :

```
good.snps <- objet1[objet1$QUAL > 99, "ID"]
length(good.snps)
#> [1] 97
```

Et si... ?

1. Charger **df.tsv** sans **header = TRUE**

```
df <- data.frame(x = 1:3, y = c(4, 5, 8), nom = c("a", "b", "c"))
write.table(df, file = "df_exemple.tsv", sep = "\t", quote = FALSE, row.names = FALSE)
sans_entete <- read.table("df_exemple.tsv", sep = "\t")
sans_entete
#>   V1 V2 V3
#> 1  x  y nom
#> 2  1  4  a
#> 3  2  5  b
#> 4  3  8  c
str(sans_entete)
```

```
#> 'data.frame': 4 obs. of 3 variables:
#> $ V1: chr "x" "1" "2" "3"
#> $ V2: chr "y" "4" "5" "8"
#> $ V3: chr "nom" "a" "b" "c"
```

La ligne d'en-tête est lue comme une ligne de données : on obtient une ligne de trop, des colonnes nommées **V1**, **V2**, **V3**, et **toutes les colonnes deviennent du texte** (une colonne ne contient qu'un type, et "x" n'est pas un nombre).

2. Chercher un SNP qui n'existe pas

```
objet1[ids == "rs000", "QUAL"]
#> integer(0)
```

Aucune erreur : le vecteur logique ne contient que des **FALSE**, donc aucune rangée n'est sélectionnée et R renvoie un vecteur **vide**. Une faute de frappe dans un identifiant passe donc inaperçue si on ne vérifie pas le résultat.

3. Oublier la virgule

```
objet1[objet1$QUAL > 99]
#> Error in `[.data.frame'`(objet1, objet1$QUAL > 99): undefined columns selected
```

Sans virgule, R interprète le vecteur logique comme une sélection de **colonnes**. Ce vecteur a une valeur par rangée, donc beaucoup plus de valeurs qu'il n'y a de colonnes : R signale des colonnes inexistantes (**undefined columns selected**).

4. Une valeur manquante dans QUAL

Démonstration sur un petit exemple :

```
snps <- data.frame(ID = c("rs1", "rs2", "rs3", "rs4"), QUAL = c(120, NA, 50, 101))
mean(snps$QUAL)
#> [1] NA
mean(snps$QUAL, na.rm = TRUE)
#> [1] 90.33333
snps[snps$QUAL > 99, "ID"]
#> [1] "rs1" NA "rs4"
snps[which(snps$QUAL > 99), "ID"]
#> [1] "rs1" "rs4"
```

- Question 5 : la moyenne devient **NA**. L'argument **na.rm = TRUE** retire les valeurs manquantes.
- Question 6 : la comparaison **NA > 99** donne **NA**, ce qui ajoute un **NA** dans **good.snps**. **which()** ne garde que les positions où la condition est **TRUE**.

